

Lab 2 Introduction of Jupyter notebook and Basic Python

In this lab, we will give a brief introduction of jupyter notebook (Colab) and basic use of python

Objectives

1. Know how to write python in jupyter notebook (Colab).
2. Know basic uses in python

1 Python

1.1 Introduction

Python is an interpreted, high-level, general-purpose programming language. Created by Guido van Rossum and first released in 1991. Python's design philosophy emphasizes code readability with its notable use of significant whitespace. Its language constructs and object-oriented approach aim to help programmers write clear, logical code for small and large-scale projects.

Python is dynamically typed and garbage-collected. It supports multiple programming paradigms, including structured (particularly, procedural), object-oriented, and functional programming. Python is often described as a “batteries included” language due to its comprehensive standard library.

Python was conceived in the late 1980s as a successor to the ABC language. Python 2.0, released in 2000, introduced features like list comprehensions and a garbage collection system capable of collecting reference cycles. Python 3.0, released in 2008, was a major revision of the language that is not completely backward-compatible, and much Python 2 code does not run unmodified on Python 3.

The Python 2 language, i.e., Python 2.7.x, was officially discontinued on 1 January 2020 (first planned for 2015) after which security patches and other improvements will not be released for it. With Python 2's end-of-life, only Python 3.5.x and later are supported.

1.2 Installation

In the labs of this course, we will conduct experiments and code in Colab. Thus we do not install Python to our local PC end.

2 Jupyter notebook (Colab)

2.1 Introduction

- Jupyter Notebook

The Jupyter Notebook is an open-source web application that allows you to create and share documents that contain live code, equations, visualizations and narrative text. Uses include: data cleaning and transformation, numerical simulation, statistical modeling, data visualization, machine learning, and much more. We normally use jupyter notebook to create simple python program.

- Google Colaboratory

Colab is a hosted Jupyter Notebook service that requires no setup to use and provides free access to computing resources, including GPUs and TPUs. Colab is especially well suited to machine learning, data science, and education.

```
[1]: # this part is for importing Google Drive to the directory
from google.colab import drive
drive.mount('/content/drive')

# lib for image display
from IPython.display import Image
```

Mounted at /content/drive

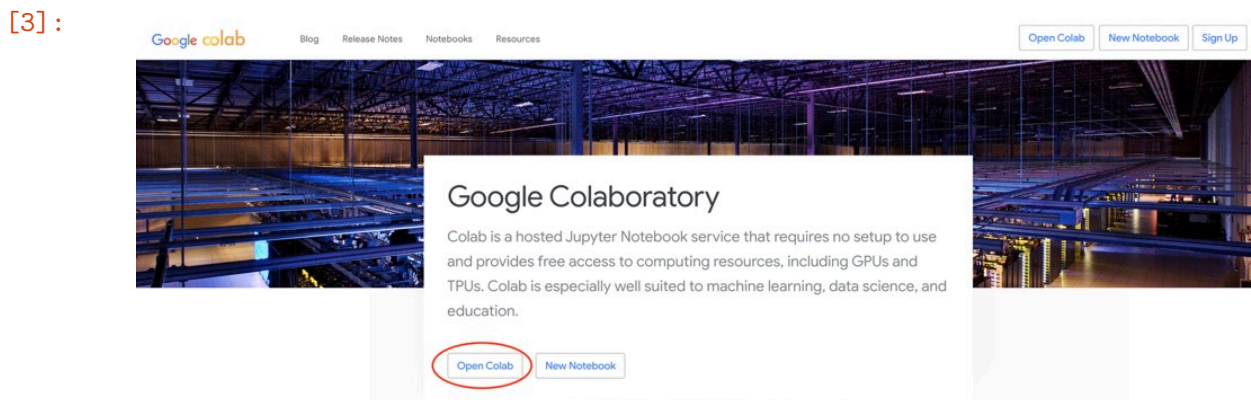
```
[2]: # show all results in one cell but not the last one
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = "all"
```

2.2 Open Colab

1. Open your browser and visit

<https://colab.google/>

```
[3]: Image(filename = "/content/drive/MyDrive/Colab Notebooks/Colab.jpg")
```

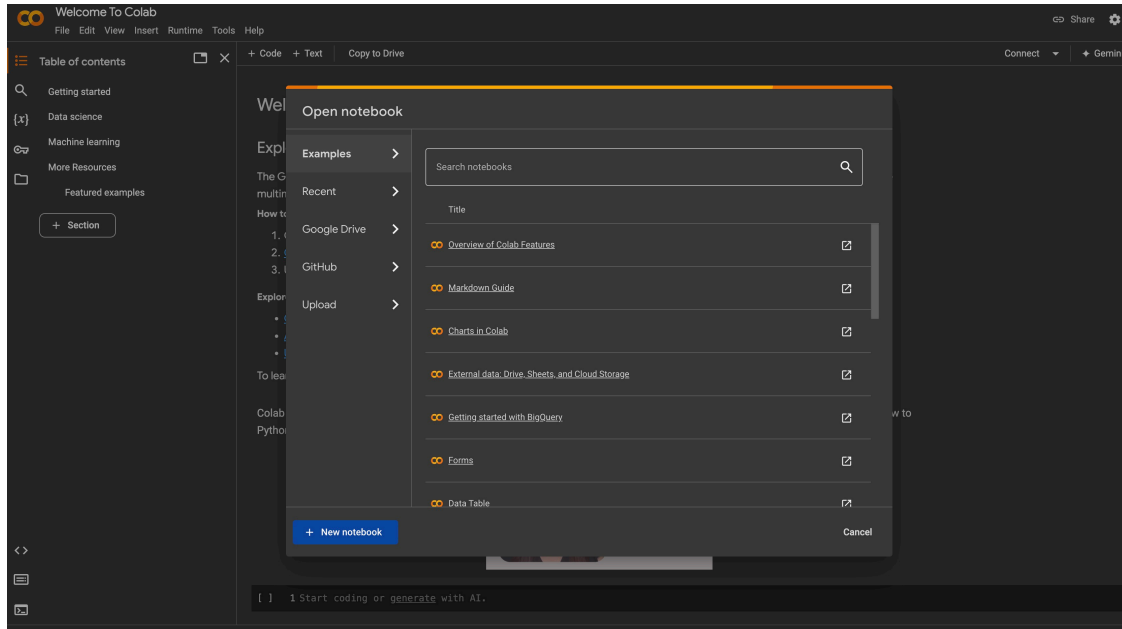


2. Click the button of

Open Colab

```
[4]: Image(filename = "/content/drive/MyDrive/Colab Notebooks/interface.jpg")
```

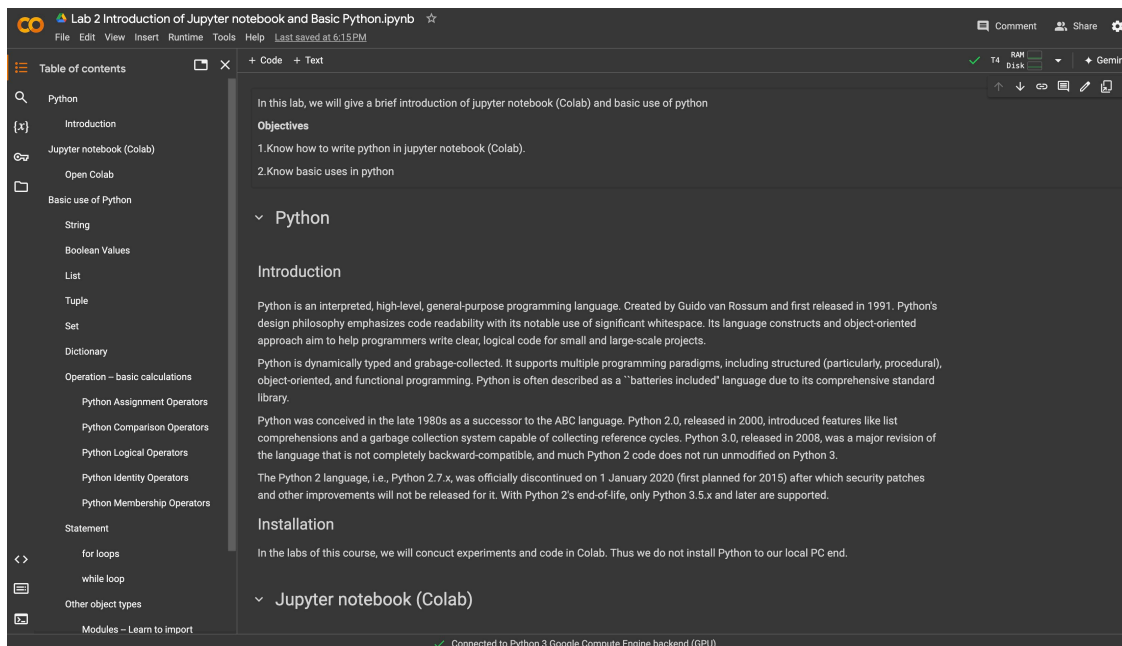
[4]:



3. Click to open the objective file and now we can edit it in Colab.

```
[5]: Image(filename = "/content/drive/MyDrive/Colab Notebooks/lab2.jpg")
```

[5]:



4. If we'd like to change the hardware for running program between cpu and gpu, we can click the tab of

Runtime

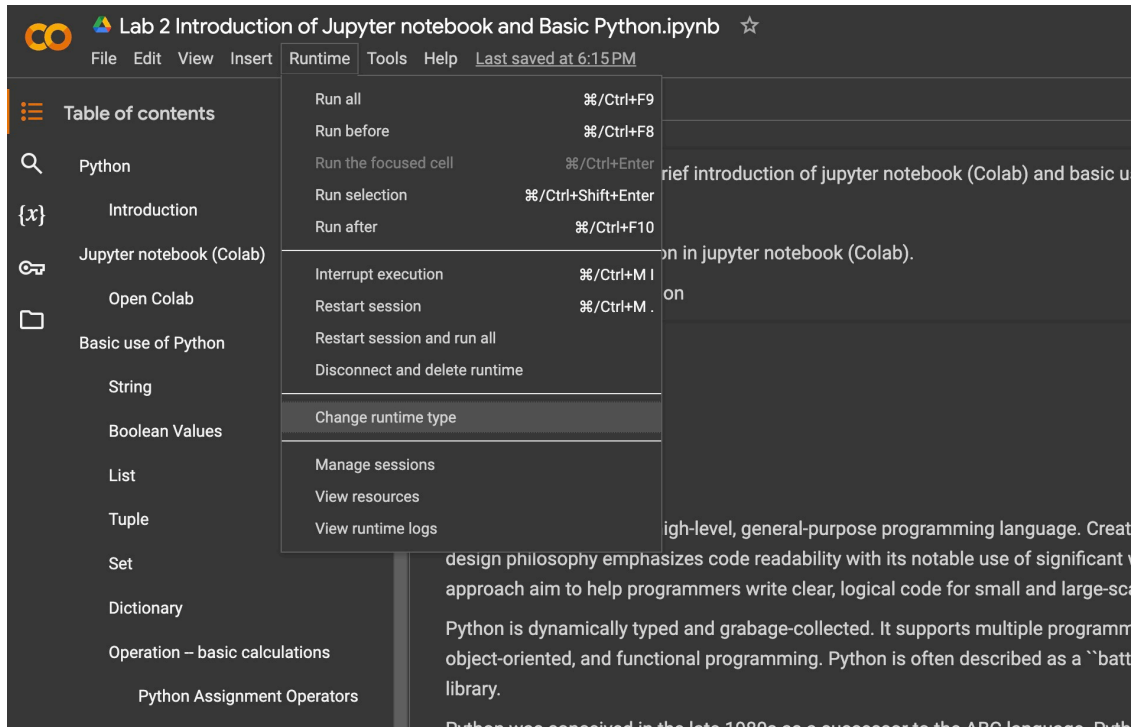
and then

Change runtime type

to choose our preferences.

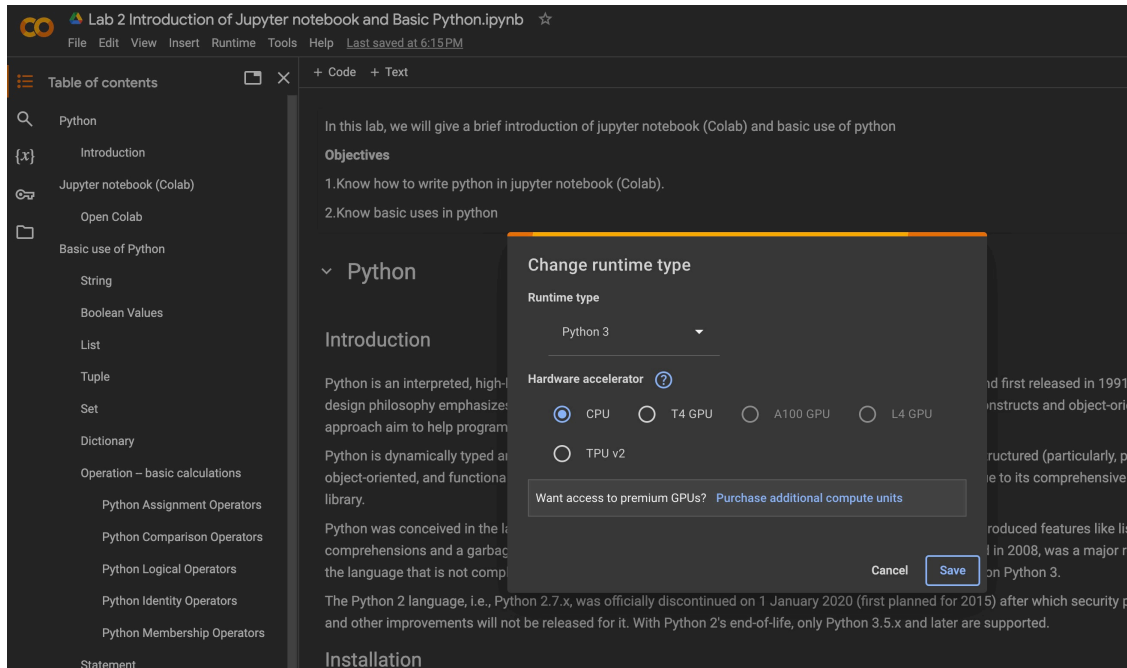
```
[6]: Image(filename = "/content/drive/MyDrive/Colab Notebooks/runtime.jpg")
```

[6]:



```
[7]: Image(filename = "/content/drive/MyDrive/Colab Notebooks/cgpu.jpg")
```

[7]:



3 Basic use of Python

3.1 Data types

Numeric Types: int, float

Text Type: str

Sequence Types: list, tuple

Mapping Type: dict

Set Types: set

Boolean Type: bool

3.1.1 Numbers

Int: Int, or integer, is a whole number, positive or negative, without decimals, of unlimited length.

```
[8]: x = 1
      y = 35656222554887711
      z = -3255522

      print(type(x))
      print(type(y))
      print(type(z))
```

```
<class 'int'>
<class 'int'>
<class 'int'>
```

Float: Float, or “floating point number” is a number, positive or negative, containing one or more decimals.

```
[9]: x = 1.10
      y = 1.0
      z = -35.59

      print(type(x))
      print(type(y))
      print(type(z))
```

```
<class 'float'>
<class 'float'>
<class 'float'>
```

3.1.2 String

String literals in python are surrounded by either single quotation marks, or double quotation marks.

‘hello’ is the same as “hello”.

You can display a string literal with the `print()` function

Assign String to Variable

Assigning a string to a variable is done with the variable name followed by an equal sign and the string:

```
[10]: a = "Hello"
      print(a)
```

Hello

Multiline Strings

You can assign a multiline string to a variable by using three quotes:

```
[11]: a = """Lorem ipsum dolor sit amet,
      consectetur adipiscing elit,
      sed do eiusmod tempor incididunt
      ut labore et dolore magna aliqua."""
      print(a)
```

```
Lorem ipsum dolor sit amet,
consectetur adipiscing elit,
sed do eiusmod tempor incididunt
ut labore et dolore magna aliqua.
```

Strings are Arrays

Like many other popular programming languages, strings in Python are arrays of bytes representing unicode characters.

However, Python does not have a character data type, a single character is simply a string with a length of 1. Square brackets can be used to access elements of the string.

```
[12]: a = "Hello, World!"  
      print(a[1])
```

e

Slicing

You can return a range of characters by using the slice syntax.

Specify the start index and the end index, separated by a colon, to return a part of the string.

```
[13]: b = "Hello, World!"  
      print(b[2:5])
```

llo

String Length

To get the length of a string, use the len() function.

```
[14]: a = "Hello, World!"  
      print(len(a))
```

13

String Methods

Python has a set of built-in methods that you can use on strings.

```
[15]: # The replace() method replaces a string with another string:  
  
a = "Hello, World!"  
print(a.replace("H", "J"))
```

Jello, World!

```
[16]: # Merge variable a with variable b into variable c:  
  
a = "Hello"  
b = "World"  
c = a + b  
print(c)
```

HelloWorld

```
[17]: # Transfer a data type to a string
a = 20
print(type(a))
a = str(a)
print(type(a))
print('I am ' + a + ' years old.')
```

```
<class 'int'>
<class 'str'>
I am 20 years old.
```

3.1.3 Boolean Values

In programming you often need to know if an expression is True or False.

You can evaluate any expression in Python, and get one of two answers, True or False.

When you compare two values, the expression is evaluated and Python returns the Boolean answer:

```
[18]: print(10 > 9)
print(10 == 9)
print(10 < 9)
```

```
True
False
False
```

3.1.4 List

A list is a collection which is ordered and changeable. In Python lists are written with square brackets.

```
[19]: # Create a List:

thislist = ["apple", "banana", "cherry"]
print(thislist)
```

```
['apple', 'banana', 'cherry']
```

```
[20]: # Print the second item of the list:

thislist = ["apple", "banana", "cherry"]
print(thislist[1])
```

```
banana
```

```
[21]: # Print the last item of the list:

thislist = ["apple", "banana", "cherry"]
print(thislist[-1])
```

cherry

[22]: *# Return the third, fourth, and fifth item:*

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[2:5])
```

['cherry', 'orange', 'kiwi']

[23]: *# This example returns the items from the beginning to "orange":*

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[:4])
```

['apple', 'banana', 'cherry', 'orange']

[24]: *# This example returns the items from "cherry" and to the end:*

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[2:])
```

['cherry', 'orange', 'kiwi', 'melon', 'mango']

[25]: *# This example returns the items from index -4 (included) to index -1 (excluded)*

```
thislist = ["apple", "banana", "cherry", "orange", "kiwi", "melon", "mango"]
print(thislist[-4:-1])
```

['orange', 'kiwi', 'melon']

[26]: *# Change the second item:*

```
thislist = ["apple", "banana", "cherry"]
thislist[1] = "blackcurrant"
print(thislist)
```

['apple', 'blackcurrant', 'cherry']

[27]: *# Print the number of items in the list:*

```
thislist = ["apple", "banana", "cherry"]
print(len(thislist))
```

3

[28]: *# Using the append() method to append an item:*

```
thislist = ["apple", "banana", "cherry"]
```

```
thislist.append("orange")
print(thislist)
```

```
['apple', 'banana', 'cherry', 'orange']
```

[29]: *# Insert an item as the second position:*

```
thislist = ["apple", "banana", "cherry"]
thislist.insert(1, "orange")
print(thislist)
```

```
['apple', 'orange', 'banana', 'cherry']
```

[30]: *# The remove() method removes the specified item:*

```
thislist = ["apple", "banana", "cherry"]
thislist.remove("banana")
print(thislist)
```

```
['apple', 'cherry']
```

[31]: *# The pop() method removes the specified index, (or the last item if index is no*

```
thislist = ["apple", "banana", "cherry"]
thislist.pop()
print(thislist)
```

[31]: 'cherry'

```
['apple', 'banana']
```

[32]: *# The del keyword removes the specified index:*

```
thislist = ["apple", "banana", "cherry"]
del thislist[0]
print(thislist)
```

```
['banana', 'cherry']
```

[33]: *# Join two list:*

```
list1 = ["a", "b", "c"]
list2 = [1, 2, 3]
list3 = list1 + list2
print(list3)
```

```
['a', 'b', 'c', 1, 2, 3]
```

```
[34]: # Using the list() constructor to make a List:

thislist = list(("apple", "banana", "cherry")) # note the double round-brackets
print(thislist)
```

```
['apple', 'banana', 'cherry']
```

3.1.5 Tuple

A tuple is a collection which is ordered and unchangeable. In Python tuples are written with round brackets.

```
[35]: # Create a Tuple:

thistuple = ("apple", "banana", "cherry")
print(thistuple)
```

```
('apple', 'banana', 'cherry')
```

```
[36]: # Print the second item in the tuple:

thistuple = ("apple", "banana", "cherry")
print(thistuple[1])
```

```
banana
```

```
[37]: # Print the last item of the tuple:

thistuple = ("apple", "banana", "cherry")
print(thistuple[-1])
```

```
cherry
```

```
[38]: # Return the third, fourth, and fifth item:

thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(thistuple[2:5])
```

```
('cherry', 'orange', 'kiwi')
```

```
[39]: # This example returns the items from index -4 (included) to index -1 (excluded)

thistuple = ("apple", "banana", "cherry", "orange", "kiwi", "melon", "mango")
print(thistuple[-4:-1])
```

```
('orange', 'kiwi', 'melon')
```

```
[40]: # Convert the tuple into a list to be able to change it:
```

```
x = ("apple", "banana", "cherry")
y = list(x)
y[1] = "kiwi"
x = tuple(y)

print(x)
```

('apple', 'kiwi', 'cherry')

```
[41]: # Print the number of items in the tuple:
thistuple = ("apple", "banana", "cherry")
print(len(thistuple))
```

3

```
[42]: # The del keyword can delete the tuple completely:
thistuple = ("apple", "banana", "cherry")
del thistuple
print(thistuple) #this will raise an error because the tuple no longer exists
```

```
-----
NameError                                Traceback (most recent call last)
<ipython-input-42-98a7419b5521> in <cell line: 4>()
      2 thistuple = ("apple", "banana", "cherry")
      3 del thistuple
----> 4 print(thistuple) #this will raise an error because the tuple no longer
    exists

NameError: name 'thistuple' is not defined
```

```
[43]: # Join two tuples:

tuple1 = ("a", "b" , "c")
tuple2 = (1, 2, 3)

tuple3 = tuple1 + tuple2
print(tuple3)
```

('a', 'b', 'c', 1, 2, 3)

```
[44]: # Using the tuple() method to make a tuple:

thistuple = tuple(("apple", "banana", "cherry")) # note the double round-bracket
print(thistuple)
```

('apple', 'banana', 'cherry')

3.1.6 Set

A set is a collection which is unordered and unindexed. In Python sets are written with curly brackets.

[45]: *# Create a Set:*

```
thisset = {"apple", "banana", "cherry"}  
print(thisset)
```

{'cherry', 'banana', 'apple'}

[46]: *# Add an item to a set, using the add() method:*

```
thisset = {"apple", "banana", "cherry"}  
  
thisset.add("orange")  
  
print(thisset)
```

{'cherry', 'banana', 'orange', 'apple'}

[47]: *# Add multiple items to a set, using the update() method:*

```
thisset = {"apple", "banana", "cherry"}  
  
thisset.update(["orange", "mango", "grapes"])  
  
print(thisset)
```

{'orange', 'grapes', 'mango', 'cherry', 'banana', 'apple'}

[48]: *# Get the number of items in a set:*

```
thisset = {"apple", "banana", "cherry"}  
  
print(len(thisset))
```

3

[49]: *# Remove "banana" by using the remove() method:*

```
thisset = {"apple", "banana", "cherry"}  
  
thisset.remove("banana")  
  
print(thisset)
```

{'cherry', 'apple'}

[50]: *# Remove the last item by using the pop() method:*

```
thisset = {"apple", "banana", "cherry"}

x = thisset.pop()

print(x)

print(thisset)
```

```
cherry
{'banana', 'apple'}
```

[51]: *# The del keyword will delete the set completely:*

```
thisset = {"apple", "banana", "cherry"}

del thisset

print(thisset)
```

```
-----
NameError                                Traceback (most recent call last)
<ipython-input-51-4e806aa89c06> in <cell line: 7>()
      5 del thisset
      6
----> 7 print(thisset)

NameError: name 'thisset' is not defined
```

[52]: *# The union() method returns a new set with all items from both sets:*

```
set1 = {"a", "b", "c"}
set2 = {1, 2, 3}

set3 = set1.union(set2)
print(set3)
```

```
{'b', 1, 'a', 'c', 2, 3}
```

[53]: *# The update() method inserts the items in set2 into set1:*

```
set1 = {"a", "b", "c"}
set2 = {1, 2, 3}

set1.update(set2)
print(set1)
```

```
{'b', 1, 'a', 'c', 2, 3}
```

```
[54]: # Using the set() constructor to make a set:
```

```
thisset = set(("apple", "banana", "cherry")) # note the double round-brackets
print(thisset)
```

```
{'cherry', 'banana', 'apple'}
```

3.1.7 Dictionary

A dictionary is a collection which is unordered, changeable and indexed. In Python dictionaries are written with curly brackets, and they have keys and values.

```
[55]: # Create and print a dictionary:
```

```
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
print(thisdict)
```

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}
```

```
[56]: # Get the value of the "model" key:
```

```
x = thisdict["model"]
print(x)
```

```
Mustang
```

```
[57]: # Get the value of the "model" key:
```

```
x = thisdict.get("model")
print(x)
```

```
Mustang
```

```
[58]: # Change the "year" to 2018:
```

```
thisdict = {
    "brand": "Ford",
    "model": "Mustang",
    "year": 1964
}
thisdict["year"] = 2018
print(thisdict)
```

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 2018}
```

```
[59]: # Print the number of items in the dictionary:
```

```
print(len(thisdict))
```

```
3
```

```
[60]: # Adding an item to the dictionary is done by using a new index key and assignin
```

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
thisdict["color"] = "red"  
print(thisdict)
```

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964, 'color': 'red'}
```

```
[61]: # The pop() method removes the item with the specified key name:
```

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
thisdict.pop("model")  
print(thisdict)
```

```
[61]: 'Mustang'
```

```
{'brand': 'Ford', 'year': 1964}
```

```
[62]: # The popitem() method removes the last inserted item (in versions before 3.7, a
```

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
thisdict.popitem()  
print(thisdict)
```

```
[62]: ('year', 1964)
```

```
{'brand': 'Ford', 'model': 'Mustang'}
```

[63]: *# The del keyword removes the item with the specified key name:*

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
del thisdict["model"]  
print(thisdict)
```

{'brand': 'Ford', 'year': 1964}

[64]: *# Make a copy of a dictionary with the dict() method:*

```
thisdict = {  
    "brand": "Ford",  
    "model": "Mustang",  
    "year": 1964  
}  
mydict = dict(thisdict)  
print(mydict)
```

{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}

[65]: *# Create a dictionary that contain three dictionaries:*

```
myfamily = {  
    "child1" : {  
        "name" : "Emil",  
        "year" : 2004  
    },  
    "child2" : {  
        "name" : "Tobias",  
        "year" : 2007  
    },  
    "child3" : {  
        "name" : "Linus",  
        "year" : 2011  
    }  
}  
print(myfamily)
```

{'child1': {'name': 'Emil', 'year': 2004}, 'child2': {'name': 'Tobias', 'year': 2007}, 'child3': {'name': 'Linus', 'year': 2011}}

[66]: *# Use the dict() constructor to make a new dictionary*
thisdict = dict(brand="Ford", model="Mustang", year=1964)
note that keywords are not string literals

```
# note the use of equals rather than colon for the assignment
print(thisdict)
```

```
{'brand': 'Ford', 'model': 'Mustang', 'year': 1964}
```

3.2 Operation – basic calculations

3.2.1 Python Arithmetic Operations

Arithmetic operators are used with numeric values to perform common mathematical operations:

Operator	Name	Example
+	Addition	$x + y$
-	Subtraction	$x - y$
	Multiplication	$x * y$
/	Division	x / y
%	Remainder	$x \% y$
	Exponentiation	$x ** y$
//	Floor division	$x // y$

```
[67]: x = 5
      y = 3

      print(x + y)
```

8

```
[68]: x = 5
      y = 3

      print(x - y)
```

2

```
[69]: x = 5
      y = 3

      print(x * y)
```

15

```
[70]: x = 12
      y = 3

      print(x / y)
```

4.0

```
[71]: x = 5
      y = 2

      print(x % y)
```

1

```
[72]: x = 2
      y = 5

      print(x ** y) #same as 2*2*2*2*2
```

32

```
[73]: x = 15
      y = 2

      print(x // y) #the floor division // rounds the result down to the nearest whole
```

7

3.2.2 Python Assignment Operators

Assignment operators are used to assign values to variables:

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
//=	x //= 3	x = x // 3
=	x **= 3	x = x ** 3

```
[74]: x = 5

      print(x)
```

5

```
[75]: x = 5
      x += 3

      print(x)
```

8

```
[76]: x = 5  
      x -= 3  
  
      print(x)
```

2

```
[77]: x = 5  
  
      x *= 3  
  
      print(x)
```

15

```
[78]: x = 5  
  
      x /= 3  
  
      print(x)
```

1.6666666666666667

```
[79]: x = 5  
  
      x %= 3  
  
      print(x)
```

2

```
[80]: x = 5  
  
      x //= 3  
  
      print(x)
```

1

```
[81]: x = 5  
  
      x **= 3  
  
      print(x)
```

125

3.2.3 Python Comparison Operators

Comparison operators are used to compare two values:

Operator	Name	Example
==	Equal	x == y
!=	Not Equal	x != y
>	Greater than	x > y
<	Less than	x < y
>=	Greater than or equal to	x >= y
<=	Less than or equal to	x <= y

```
[82]: x = 5
      y = 3

      print(x == y)

      # returns False because 5 is not equal to 3
```

False

```
[83]: x = 5
      y = 3

      print(x != y)

      # returns True because 5 is not equal to 3
```

True

```
[84]: x = 5
      y = 3

      print(x > y)

      # returns True because 5 is greater than 3
```

True

```
[85]: x = 5
      y = 3

      print(x < y)

      # returns False because 5 is not less than 3
```

False

```
[86]: x = 5
      y = 3
      print(x >= y)
      # returns True because five is greater, or equal, to 3
```

True

```
[87]: x = 5
      y = 3
      print(x <= y)
      # returns False because 5 is neither less than or equal to 3
```

False

3.2.4 Python Logical Operators

Logical operators are used to combine conditional statements:

Operator	Description	Example
and	Returns True if both statements are true	$x < 5$ and $x < 10$
or	Returns True if one of the statements is true	$x < 5$ or $x < 4$
not	Reverse the result, returns False if the result is true	$\text{not}(x < 5 \text{ and } x < 10)$

```
[88]: x = 5
      print(x > 3 and x < 10)
      # returns True because 5 is greater than 3 AND 5 is less than 10
```

True

```
[89]: x = 5
      print(x > 3 or x < 4)
      # returns True because one of the conditions are true (5 is greater than 3, but
```

True

```
[90]: x = 5
      print(not(x > 3 and x < 10))
      # returns False because not is used to reverse the result
```

False

3.2.5 Python Identity Operators

Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location:

Operator	Description	Example
is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y

```
[91]: x = ["apple", "banana"]
      y = ["apple", "banana"]
      z = x

      print(x is z)
      # returns True because z is the same object as x

      print(x is y)
      # returns False because x is not the same object as y, even if they have the
      ↪ same

      print(x == y)
      # to demonstrate the difference between "is" and "==": this comparison returns
```

```
True
False
True
```

```
[92]: x = ["apple", "banana"]
      y = ["apple", "banana"]
      z = x

      print(x is not z)
      # returns False because z is the same object as x

      print(x is not y)
      # returns True because x is not the same object as y, even if they have the same

      print(x != y)
      # to demonstrate the difference between "is not" and "!=": this comparison retu
```

```
False
True
False
```

3.2.6 Python Membership Operators

Membership operators are used to test if a sequence is presented in an object:

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object	x not in y

```
[93]: x = ["apple", "banana"]
      print("banana" in x)
      # returns True because a sequence with the value "banana" is in the list
```

True

```
[94]: x = ["apple", "banana"]
      print("pineapple" not in x)
      # returns True because a sequence with the value "pineapple" is not in the list
```

True

3.3 Statement

3.3.1 if else

Python conditions and if statements

Python supports the usual logical conditions from mathematics:

Equals: a == b

Not equals: a != b

Less than: a < b

Less than or equal to: a <= b

Greater than: a > b

Greater than or equal to: a >= b

These conditions can be used in several ways, most commonly in “if statements” and loops.

An “if statement” is written by using the if keyword.

```
[95]: a = 33
      b = 200
      if b > a:
          print("b is greater than a")
```

b is greater than a

Elif

The elif keyword is python's way of saying "if the previous conditions were not true, then try this condition".

```
[96]: a = 33
      b = 33
      if b > a:
          print("b is greater than a")
      elif a == b:
          print("a and b are equal")
```

a and b are equal

Else

The else keyword catches anything which isn't caught by the preceding conditions.

```
[97]: a = 200
      b = 33
      if b > a:
          print("b is greater than a")
      elif a == b:
          print("a and b are equal")
      else:
          print("a is greater than b")
```

a is greater than b

3.3.2 for loops

A for loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string). This is less like the for keyword in other programming languages, and works more like an iterator method as found in other object-oriented programming languages. With the for loop we can execute a set of statements, once for each item in a list, tuple, set etc.

Print each fruit in a fruit list:

```
[98]: fruits = ["apple", "banana", "cherry"]
      for x in fruits:
          print(x)
```

apple
banana
cherry

Loop through the letters in the word "banana":

```
[99]: for x in "banana":
      print(x)
```

b
a
n
a
n
a

The break Statement

With the break statement we can stop the loop before it has looped through all the items:

```
[100]: fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    print(x)  
    if x == "banana":  
        break
```

apple
banana

The continue Statement

With the continue statement we can stop the current iteration of the loop, and continue with the next:

```
[101]: fruits = ["apple", "banana", "cherry"]  
for x in fruits:  
    if x == "banana":  
        continue  
    print(x)
```

apple
cherry

3.3.3 while loop

With the while loop we can execute a set of statements as long as a condition is true.

```
[102]: i = 1  
while i < 6:  
    print(i)  
    i += 1
```

1
2
3
4
5

3.4 Other object types

3.4.1 Functions

A function is a block of code which only runs when it is called. You can pass data, known as parameters, into a function. A function can return data as a result.

Creating a Function

In Python a function is defined using the `def` keyword:

```
[103]: def my_function():  
        print("Hello from a function")
```

Calling a Function

To call a function, use the function name followed by parenthesis:

```
[104]: def my_function():  
        print("Hello from a function")  
  
my_function()
```

Hello from a function

Arguments

Information can be passed into functions as arguments. Arguments are specified after the function name, inside the parentheses. You can add as many arguments as you want, just separate them with a comma. The following example has a function with one argument (fname). When the function is called, we pass along a first name, which is used inside the function to print the full name:

```
[105]: def my_function(fname):  
        print(fname + " Refsnes")  
  
my_function("Emil")  
my_function("Tobias")  
my_function("Linus")
```

Emil Refsnes

Tobias Refsnes

Linus Refsnes

3.4.2 Modules – Learn to import from modules

What is a Module?

Consider a module to be the same as a code library. A file containing a set of functions you want to include in your application.

Create a Module

To create a module just save the code you want in a file with the file extension `.py`:

```
[110]: # Save this code in a file named mymodule.py
%%writefile mymodule.py

def greeting(name):
    print("Hello, " + name)

person1 = {
    "name": "John",
    "age": 36,
    "country": "Norway"
}
```

Overwriting mymodule.py

Use a Module

Now we can use the module we just created, by using the import statement:

```
[111]: import mymodule

mymodule.greeting("Jonathan")
```

Hello, Jonathan

Import From Module

You can choose to import only parts from a module, by using the from keyword.

```
[112]: #The module named mymodule has one function and one dictionary:
from mymodule import person1

print (person1["age"])
```

36

4 Conclusion

In this lab, we learnt:

1. What is python;
2. What is Jupyter notebook and Colab;
3. How to strat Colab and edit/program in it;
4. Variable types and basic operations in python.

```
[ ]:
```

```
[ ]:
```

```
[ ]:
```